MMDU MULLANA OFFICIAL SYLLABUS • BCSE-008 • UNIT 1

Unit 1: Programming Fundamentals, Compilation Pipeline & Problem Solving

Subject: Computational & Problem Solving Using C (BCSE-008) • **Exam Weightage:** 24 - 28 Marks • **Study Time:** 40 mins

🗺️ UNIT 1 SYLLABUS ROADMAP & TOPIC INDEX (TABLE OF CONTENTS)

6 Modules • 16 Topics • Pages 1–11

1. 1. Computational Problem Classification & Logic Types — Pg. 1–2

- TOPIC 1. Computational Problem Classifications — Pg. 1
- TOPIC 2. Logic Paradigms: Deductive vs. Inductive Reasoning — Pg. 1

2. 2. Algorithmic Design, Flowcharts & Stepwise Refinement — Pg. 2–3

- TOPIC 1. Stepwise Refinement (Top-Down Decomposition) — Pg. 2
- TOPIC 2. Standard ANSI Flowcharting Symbols — Pg. 2

3. 3. Testing Strategies & 4 Tiers of Program Defects — Pg. 3–4

- TOPIC 1. Black-Box vs. White-Box Testing Methodologies — Pg. 3
- TOPIC 2. The 4 Tiers of Programming Defects — Pg. 3

4. 4. Algorithmic Complexity (Big-O) & Dry-Run Trace Tables — Pg. 4–5

- TOPIC 1. Asymptotic Complexity: Big-O Mathematical Taxonomy — Pg. 4
- TOPIC 2. Dry-Run Variable Execution Trace Table — Pg. 4

5. 5. Translators, 4-Stage Compilation Pipeline & Process Memory Layout — Pg. 5–6

- TOPIC 1. Translators: Compiler vs. Interpreter vs. Assembler — Pg. 5
- TOPIC 2. The 4-Stage C Compilation Pipeline — Pg. 5
- TOPIC 3. C Process Virtual Memory Layout in RAM — Pg. 5

6. 6. University Examination Solved Questions (Unit 1) — Pg. 6–11

- TOPIC University Solved Examination Problems (MMDU Pattern) — Pg. 6
- TOPIC Question 1 (1 Mark): Missing Intermediate Stage — Pg. 6
- TOPIC Question 2 (1 Mark): Memory Segment Identification — Pg. 6
- TOPIC Question 13 (2 Marks): Dry Run Loop Trace — Pg. 7
- TOPIC Question 17 (4 Marks): Quadratic Equation Algorithm & Flowchart — Pg. 7

1. Computational Problem Classification & Logic Types

1. Computational Problem Classifications

In computer science and software engineering, computational problems are systematically categorized based on the nature of their output space:

1. **Decision Problems:** Problems requiring a deterministic Boolean output (`True` or `False` , `1` or `0`). Example: Given an integer N, decide whether N is prime.

2. **Counting Problems:** Problems that compute the cardinality of valid solutions satisfying specific constraints. Example: Counting the number of distinct paths in a grid.

3. **Optimization Problems:** Problems requiring the identification of the globally best candidate among all feasible configurations according to an objective function. Example: Shortest path routing in network packet switching.

---

2. Logic Paradigms: Deductive vs. Inductive Reasoning

Program construction and formal algorithmic proofs rely on two core logic frameworks:

#	Dimension	Deductive Logic	Inductive Logic
1	**Direction**	**Top-Down:** Starts with general axioms/theories and deduces specific guaranteed truths.	**Bottom-Up:** Starts with specific empirical observations and infers general probabilistic patterns.
2	**Validation Guarantee**	**Guaranteed Absolute Truth:** If premises are valid and logic rules hold, conclusions are incontrovertible.	**Probabilistic / Plausible:** True for observed training sample, but unverified for unobserved edge cases.
3	**Software Role**	Formal program verification, loop invariants, and algorithmic correctness proofs.	Test-driven development (TDD), heuristic AI search, and empirical profiling.

| **4** | **C Engineering Example** | Proving loop invariant: If loop starts at `i=0` and increments by 1, `while(i < N)` executes exactly N iterations. | Running 10 unit test cases with positive inputs: it suggests the code works, but does not mathematically prove absence of memory leaks. |

2. Algorithmic Design, Flowcharts & Stepwise Refinement

1. Stepwise Refinement (Top-Down Decomposition)

Stepwise Refinement (formulated by Niklaus Wirth) is an architectural discipline where a high-level problem statement is iteratively decomposed into smaller, self-contained sub-modules until every sub-task maps directly to elementary language primitives.

✅ **FUNDAMENTAL LAW / RULE:**
Core Benefits of Stepwise Refinement:

- **Cognitive Manageability:** Prevents cognitive overload by isolating architectural concerns.

- **Modularity:** Subroutines can be developed, compiled, and unit-tested in isolation.

- **Reusability:** Common computational kernels (e.g., searching, sorting) become shared utility functions.

---

2. Standard ANSI Flowcharting Symbols

ANSI standardized flowchart symbology enforces unambiguous visual algorithmic blueprints:

Symbol Shape	ANSI Geometric Form	Engineering Purpose	C Language Mapping
Terminal	Rounded Oval / Capsule	Indicates entry point (Start) or termination point (Stop/End).	`int main()` / `return 0;`
Input / Output	Parallelogram	Data ingestion or presentation across peripheral streams.	`scanf()`, `printf()`, `fgets()`
Process	Rectangle	Computational operations, variable initializations, state mutations.	`c = a + b;`, `i = 0;`
Decision	Rhombus (Diamond)	Conditional evaluation producing branching Boolean pathways (True/False).	`if (x > 0)`, `switch (ch)`

| **Connector** | Circle with alphanumeric key | Merges convergent execution paths without crossing connector arrows. | Loop junction nodes |

3. Testing Strategies & 4 Tiers of Program Defects

1. Black-Box vs. White-Box Testing Methodologies

Quality assurance in compiled systems incorporates two complementary testing paradigms:

- **Black-Box Testing (Functional Testing):**

- Assesses software behavior purely against specification inputs and expected outputs.

- The tester has **zero visibility** into the internal source code architecture.
- Primary Techniques: **Equivalence Class Partitioning** and **Boundary Value Analysis (BVA)**.

- **White-Box Testing (Structural Testing):**

- Inspects internal control structures, branches, and execution paths with full source code visibility.
- Primary Metrics: **Statement Coverage**, **Branch/Decision Coverage**, and **Cyclomatic Complexity**.

---

2. The 4 Tiers of Programming Defects

Software defects are classified into four distinct phases of the software lifecycle:

Error Tier	Detection Phase	Root Cause & Characteristics	C Concrete Example
1. Syntax Error	Translation / Compile Time	Violation of C grammar rules; caught by compiler tokenizer or parser.	Missing semicolon `;`, mismatched braces `{`, misspelled keyword `whlie`.
2. Linker Error	Linking Time (`ld`)	Unresolved external symbols; function declared but missing object definition.	Calling `clrscr()` or `foo()` without linking their object library.
3. Runtime Error	Program Execution Time	Illegal CPU instructions or OS protection faults causing abnormal termination.	Division by zero (`x / 0`), Segmentation Fault (SIGSEGV) accessing address 0x0.

| **4. Semantic / Logic Error** | User Testing / Output Validation | Program compiles and runs without crashing, but generates incorrect results. | Using `i <= n` instead of `i < n` causing an off-by-one array overwrite. |

4. Algorithmic Complexity (Big-O) & Dry-Run Trace Tables

1. Asymptotic Complexity: Big-O Mathematical Taxonomy

Big-O notation characterizes the upper bound of resource consumption (time or memory space) as input size N tends to infinity:

Notation	Complexity Name	Growth Rate for $N = 1000$	Canonical C Algorithm
O(1)	Constant Time	1 operation	Array indexing `A[i]`, pointer dereference `*p`.
O(log N)	Logarithmic Time	≈ 10 operations	Binary search in sorted array (halves problem space each step).
O(N)	Linear Time	1,000 operations	Linear sequential search, single-pass array traversal.
O(N log N)	Linearithmic Time	$\approx 10,000$ operations	Merge Sort, Quick Sort (average case divide & conquer).
O(N^2)	Quadratic Time	1,000,000 operations	Bubble Sort, Selection Sort, nested 2D matrix iteration.

2. Dry-Run Variable Execution Trace Table

A **Dry Run** is a manual code simulation tracing variable states against line numbers to expose logic defects:

```
int sum = 0;
for (int i = 1; i <= 3; i++) {
    sum += i;
}
```

Trace Table:

Step	Source Line	Variable `i`	Condition `i <= 3`	Variable `sum`	Step Action Summary
0	Line 1	Undefined	-	0	Initialization of accumulator `sum = 0`.
1	Line 2	1	1 <= 3 (True)	0	Loop init `i = 1`; entry condition holds.
2	Line 3	1	-	1	Accumulate: `sum = 0 + 1 = 1`.
3	Line 2	2	2 <= 3 (True)	1	Increment `i++`; condition holds.
4	Line\frac{3}{2}	-	3	Accumulate: `sum = 1 + 2 = 3`.	
5	Line 2	3	3 <= 3 (True)	3	Increment `i++`; condition holds.
6	Line 3	3	-	6	Accumulate: `sum = 3 + 3 = 6`.

| **7** | Line 2 | 4 | 4 <= 3 (False) | 6 | Increment `i++`; condition fails ➔ Loop terminates! |

5. Translators, 4-Stage Compilation Pipeline & Process Memory Layout

1. Translators: Compiler vs. Interpreter vs. Assembler

Comparative Dimension	Compiler	Interpreter	Assembler
Input Source	High-level language (C, C++, Rust).	High-level script (Python, JS, Ruby).	Assembly language mnemonics (8086, x86).
Translation Unit	Scans entire translation unit at once.	Interprets statement-by-statement.	1-to-1 conversion of mnemonic to opcode.
Intermediate Object	Generates relocatable object file (`.o` / `.obj`).	No standalone object file produced.	Generates relocatable machine code.
Execution Speed	Extremely fast direct hardware execution.	Slower due to runtime interpretation overhead.	Maximum execution speed on target CPU.
Error Reporting	Compiles list of all syntax errors at once.	Halts execution immediately on first error.	Reports syntax errors during assembly pass.

---

2. The 4-Stage C Compilation Pipeline

When invoking `gcc prog.c -o prog.exe`, four distinct system subsystems execute in pipeline order:

```
[ prog.c ] (Source Text)
    |
    ▼ (1. Preprocessor: cpp)
[ prog.i ] (Expanded Pure C: stripped comments, expanded #include, macro constants)
    |
    ▼ (2. Compiler: ccl)
[ prog.s ] (Assembly Code: architecture-specific target CPU instructions)
    |
    ▼ (3. Assembler: as)
[ prog.o / .obj ] (Relocatable Machine Binary: binary CPU opcodes with unresolved addresses)
    |
    ▼ (4. Linker: ld + libc.a / CRT)
[ prog.exe / a.out ] (Executable Binary: fully resolved memory addresses, ready to load into RAM)
```

---

3. C Process Virtual Memory Layout in RAM

When an executable binary is launched by the OS loader, it is allocated a private virtual memory address space:

Memory Segment	Growth Direction	Stored Content & Characteristics
Stack Segment	**Grows Downwards (High ➔ Low)**	Local automatic variables (`auto`), function parameter lists, return addresses, and stack activation records. Automatically allocated and released.
Free Pool Boundary	Stack/Heap Interface	Memory area where stack and heap grow toward each other. Depletion causes Stack Overflow.
Heap Segment	**Grows Upwards (Low ➔ High)**	Dynamically allocated memory at runtime via `malloc()` , `calloc()` , `realloc()` . Must be explicitly released via `free()` .
BSS Segment	Static / Fixed	Uninitialized global and static variables. Automatically initialized to zero by the OS loader prior to `main()` .
Initialized Data	Static / Fixed	Global and static variables explicitly initialized with non-zero literals in source code (e.g., `int global_x = 100;`).

| **Text / Code Segment** | Static / Low Memory (0x0) | Read-only compiled CPU binary instructions. Protected by OS from runtime overwrite attempts. |

6. University Examination Solved Questions (Unit 1)

University Solved Examination Problems (MMDU Pattern)

Question 1 (1 Mark): Missing Intermediate Stage

Question: In the standard C compilation pipeline, complete the missing block `[ ? ]` generated by compiler `ccl` prior to machine code generation:

```
prog.c ➔ Preprocessor (cpp) ➔ prog.i ➔ Compiler (ccl) ➔ [ ? ] ➔ Assembler (as) ➔ prog.o
```

Step-by-Step Solution:

```
prog.i (Expanded C) ➔ Compiler (ccl) ➔ Assembly Code (prog.s) ➔ Assembler (as) ➔ prog.o
```

The compiler `ccl` translates expanded pure C code into target ISA **Assembly Language Code (`prog.s`)**.

Question 2 (1 Mark): Memory Segment Identification

Question: Refer to the virtual RAM architecture below. Which labeled segment corresponds to memory allocated via `malloc()` ?

```
[ High Memory: Stack (↓) ] ➔ [ Free Memory Space ] ➔ [ Segment X (↑) ] ➔ [ BSS ] ➔ [ Data ] ➔ [ Text ]
```

Step-by-Step Solution:

```
Segment X = Heap Segment (Grows Upwards towards higher memory addresses)
```

Segment X is the Heap Segment, managed dynamically at runtime using functions in .

Question 13 (2 Marks): Dry Run Loop Trace

Question: Trace the variable states during execution of the digit-sum loop for `N = 345`:

```
while(n > 0) { rem = n % 10; sum += rem; n /= 10; }
```

Step-by-Step Solution:

Iteration	Initial `n`	Condition `n > 0`	`rem = n % 10`	`sum += rem`	Updated `n /= 10`
Init	345	-	-	0	345
Pass 1	345	True	5	0 + 5 = 5	34
Pass 2	34	True	4	5 + 4 = 9	3
Pass 3	3	True	3	9 + 3 = 12	0
Exit	0	False (Terminates)	-	**12 (Final Result)**	0

Question 17 (4 Marks): Quadratic Equation Algorithm & Flowchart

Question: Construct an algorithm and top-down decision structure to compute all roots of $ax^2 + bx + c = 0$.

Step-by-Step Solution:

1. Ingest coefficients a, b, c.

2. Compute Discriminant $D = b^2 - 4ac$.

3. Decision Branches:

- If $D > 0$: Real & Distinct roots: $r_1 = \frac{-b+sqrtD}{2a}, r_2 = \frac{-b-sqrtD}{2a}$.

- If $D = 0$: Real & Equal roots: $r_1 = r_2 = \frac{-b}{2a}$.
- If $D < 0$: Complex Conjugate roots: Real $= \frac{-b}{2a}$ Imag $= \frac{sqrt-D}{2a}$. Output: Real $\pm\, i \cdot$ Imag.

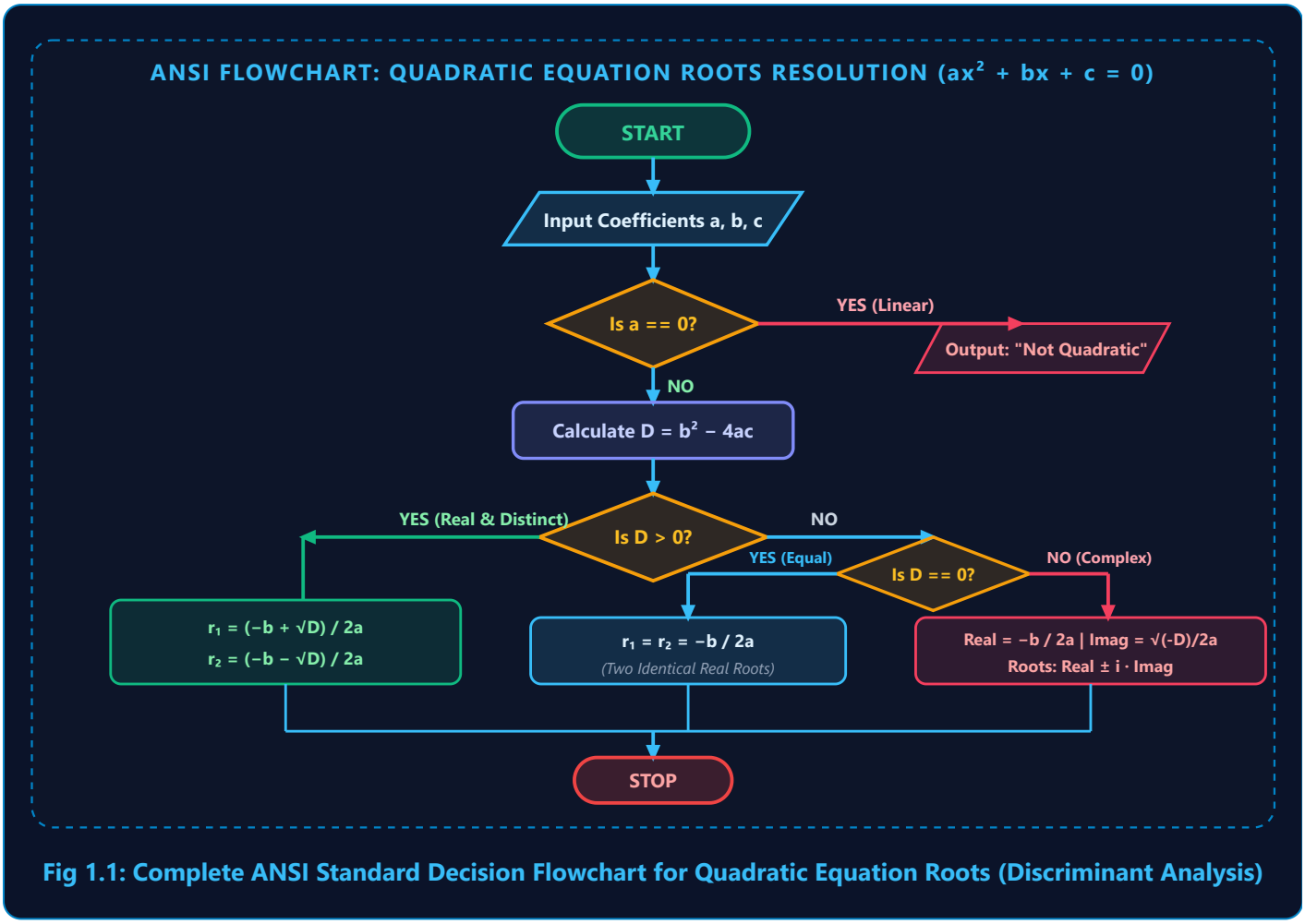


Fig 1.1: Complete ANSI Standard Decision Flowchart for Quadratic Equation Roots (Discriminant Analysis)

```
#include <stdio.h>
#include <math.h>

void solveQuadratic(double a, double b, double c) {
    if (a == 0) { printf("Not quadratic!
"); return; }
    double d = b*b - 4*a*c;
    if (d > 0) {
        printf("Roots: %.2f, %.2f
", (-b + sqrt(d))/(2*a), (-b - sqrt(d))/(2*a));
    } else if (d == 0) {
        printf("Equal Roots: %.2f
", -b/(2*a));
    } else {
        printf("Complex: %.2f + %.2fi, %.2f - %.2fi
", -b/(2*a), sqrt\frac{-d}{2*a}, -b/(2*a), sqrt\frac{-d}{2*a});
    }
}
```